How Reddit Works

Software Engineering & Design Principles

Introduction: Building a Social News Aggregator

What it is: A network of communities where people can dive into their interests, hobbies, and passions. It operates as a social news aggregation, web content rating, and discussion website with both free (ad-supported) and premium (Reddit Premium) experiences.

Scale: Millions of active users participate in hundreds of thousands of active communities (subreddits), voting on content simultaneously.

Challenge: Keeping the system highly available, serving real-time feeds, rendering deep nested comment trees, and preventing spam at a massive scale.

Our focus: The software engineering side, explained with architectural flowcharts, interaction models, and system diagrams.

> **Key idea: A platform handling millions of votes per minute requires asynchronous processing, aggressive caching, and a highly decoupled architecture.**

Process Model: Agile

- **Short cycles:** Engineering work is conducted in small, focused iterations known as sprints (typically 2 weeks).
- **Flexible:** User behavior dictates product direction. Requirements can shift rapidly based on trending events or community feedback.
- **Frequent delivery:** Working software is shipped behind feature flags, allowing gradual rollouts to specific subreddits or user cohorts.
- **Feedback:** Telemetry, upvote engagement, and community moderation metrics heavily influence what features get expanded or rolled back.

Why Agile? A social platform is defined by its users. The engineering plan must adapt quickly to how communities utilize new features (like live audio rooms or real-time typing indicators).

Requirements: Actors and Use Cases

To design the system, we first identify the primary entities (actors) outside the system boundaries and the goals (use cases) they want to achieve. This shapes our backend service boundaries.

Actor	Primary Use Cases	Description
Lurker (Unregistered)	Search, View Feed, Read Comments	Interacts purely in a read-only capacity. Served mostly from caches.
Registered User (Riya)	Post, Upvote/Downvote, Comment	Core contributor. Actions trigger database writes and feed recalculations.
Premium User (Aman)	Browse Ad-Free, Award Posts	Subscribes for enhanced features. Requires billing system integration.
Moderator (Kabir)	Approve/Remove Posts, Ban Users	Manages community guidelines. High-privilege write operations within a subreddit.
Admin (System)	Manage Site, Global Bans	Reddit employees managing global infrastructure and safety.
Ad Network Gateway	Serve Targeted Ads	External system integrated to monetize free-tier traffic.

Process Example: Shipping a Feature

Consider the feature *"Live Comment Count"*. It moves through the Agile lifecycle:

1. Backlog
Story created to show live updates.

2. Sprint
Squad pulls story, writes backend WebSocket code.

3. Rollout
Released to 5% of web users behind a feature flag.

4. Metrics
Analyze if server load is acceptable and engagement rises.

Design: Architecture and Components

Reddit transitioned from a monolithic architecture to a robust microservices ecosystem to handle scale.

- **Clients:** iOS, Android, and Web applications communicate with the backend via GraphQL APIs.
- **API Gateway:** Acts as the front door, routing requests, handling rate limiting, and terminating SSL.
- **Services:** The backend is divided into specialized domains (e.g., Feed Service, Voting Service, Search Service).
- **Data & Cache:** PostgreSQL handles durable data. Memcached/Redis handles extreme read loads (crucial for popular threads).
- **CDN:** Fastly caches static assets, images, and videos close to the end user.



Why microservices? A massive traffic spike in one subreddit (e.g., breaking news) might overload the Comment Service, but the Auth and Search services remain completely unaffected, keeping the broader site stable.

Activity Flow: Upvoting a Post

Example: Riya sees a post about space. She decides to upvote it. The system must verify her state before processing the action.

1. View Post
Client renders feed from Cache.

2. Click Upvote
Client intercepts click.

3. Decision: Logged In?
Check Auth token.

4a. Yes -> Vote
Send async vote event to backend.

4b. No -> Intercept
Prompt user with login/signup modal.

Sequence Flow: Loading a Subreddit Feed

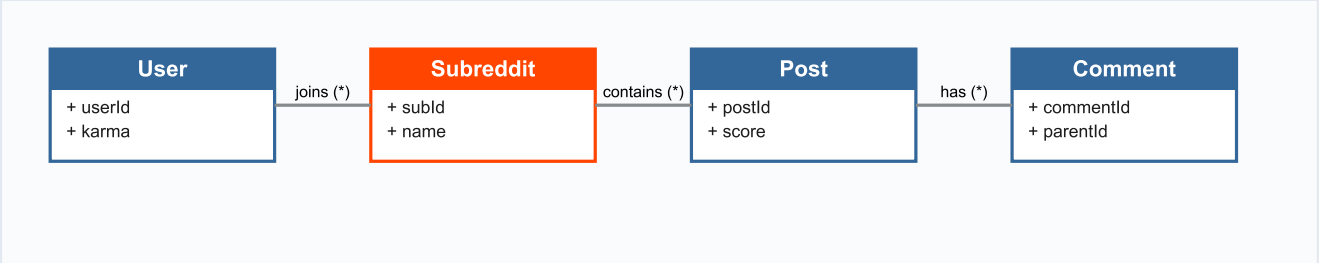
When Riya navigates to `r/science`, a sequence of backend calls executes to assemble the feed optimally.

Component	Action / Data Flow
1. Web Client	Requests `/r/science/hot` via API Gateway.
2. API Gateway	Routes request to **Feed Service**.
3. Feed Service	Checks **Redis Cache** for a pre-computed "Hot" list of post IDs for this subreddit.
4. Cache Layer	*(Cache Hit)* Returns top 25 Post IDs immediately.
5. Metadata Service	Fetches post titles, author names, and vote counts for those specific 25 IDs.
6. Web Client	Receives JSON payload, renders feed. Images stream directly from the **CDN**.

Optimization: If Riya is logged in, the Feed Service also cross-references her specific user ID to see which of those 25 posts she has already upvoted, highlighting the arrows orange.

Core Data Model

At the center of the platform are Users, Subreddits, Posts, and Comments. Their relational mapping defines how content is linked and retrieved.



Data Pipeline: From a Click to an Insight

When a user upvotes a post, it doesn't just increment a number; it feeds into a massive data pipeline that shapes the entire ecosystem.

1. User Action

Riya upvotes a post on the mobile app.

2. Event Stream

Action logged to Kafka (message broker).

3. Stream Process

Algorithm recalculates the post's "Hotness" score instantly.

4. Batch Process

Overnight jobs update Riya's personal recommendation models.

Release Flow: Shipping & Experimenting

To avoid breaking the site for millions of concurrent users, deployments follow a strict, automated pipeline with constant metric evaluation.

Stage	Description
1. Code Commit	Engineer merges a pull request to the main branch.
2. CI/CD Build	Automated tests execute. If tests fail, the build is rejected.
3. Staging	Deployed to a mirrored internal environment for QA testing.
4. Canary Release	New code routes to 1% of live traffic. Logs monitored for errors.
5. A/B Testing	Feature exposed to 5% of users. Do engagement metrics improve?
6. Decision	If metrics are positive: Full Rollout. If negative: Automatic Rollback.

Key Takeaways

- **Process:** Agile methodologies keep iterations fast and responsive to community shifts.
- **Testing:** Canary releases and heavy feature-flagging mitigate the risk of bringing down massive subreddits.
- **Requirements:** Clear use cases separate read-heavy users (lurkers) from write-heavy users (posters/mods).
- **Design:** Microservices isolate failures, while aggressive caching (Redis) prevents database meltdowns during viral events.
- **Improvement:** Every upvote and comment is streamed into a data pipeline to refine sorting algorithms continuously.

> **In one line: Isolate services to scale, cache aggressively to survive, and use community data to evolve.**